

AI-Powered Network Traffic Anomaly Detector

A Random Forest classifier trained on CICIDS2017, wired to a live Scapy capture pipeline

CPAN226: Network Programming

Final Project Report, Winter/Summer 2026

Gustavo

Individual project. Python: AI & Automation track, option 1.

Abstract

This project is an anomaly detector that captures live IP traffic with Scapy, aggregates packets into 5-tuple flows, and scores each flow with a Random Forest classifier trained on the CICIDS2017 dataset. Alerts appear on a Streamlit dashboard, refreshed every second from a WAL-mode SQLite database that the capture pipeline writes to.

On CICIDS2017's held-out test set the classifier reaches 99.96% accuracy with a macro F1 of 0.9996 across three classes (benign, DDoS, PortScan). Training takes under three seconds on a laptop CPU.

Moving from the trained model to live Windows loopback traffic surfaced an engineering problem worth documenting. The calibrated alert threshold (0.1322, for a 0.1% false-positive rate on CICIDS2017 validation benign) was too tight for the runtime environment. Synthetic attack flows produced an attack probability around 0.08, below the threshold, so nothing raised an alert. Lowering the runtime threshold to 0.05 recovered detection, while the calibrated value stayed in metadata.json for audit.

The body of the report covers the architecture, feature-engineering decisions (including the deliberate exclusion of destination_port as a leakage-prone feature), the runtime pipeline's thread topology, and three findings about model behaviour under Windows loopback that a textbook would not cover.

1. Introduction

Signature-based intrusion detection has been the workhorse of enterprise networks for two decades. The 2026 landscape has shifted toward ML-driven Network Detection and Response (NDR) systems that learn what normal traffic looks like and flag deviations. This project implements a small version of that pattern: a classifier trained on public attack data, plugged into a real-time packet capture pipeline.

The project satisfies the Python: AI & Automation track of CPAN226 (option 1, AI-Powered Network Traffic Anomaly Detector). Two attacks are covered: a volumetric SYN flood (the canonical DDoS primitive) and a TCP port scan (standard reconnaissance). Both exercise distinct packet-level signatures that a supervised classifier can learn.

The code runs end-to-end on Windows 11 with Npcap installed and a Python 3.11 virtual environment. All attack traffic stays on 127.0.0.1 to avoid any off-host impact.

2. System Architecture

The system splits into two phases. The offline phase lives in four Jupyter notebooks and produces three artifacts that the runtime consumes: detector.pkl (the fitted Random Forest), metadata.json (feature order, class map, calibrated threshold), and feature_names.json (the naming contract between training and runtime).

The online phase is four Python scripts: flow_builder.py, detector.py, capture.py, and dashboard.py, plus attack_simulator.py for demos. capture.py runs a three-thread pipeline designed so that a slow inference step cannot block packet capture.

2.1 Three-thread runtime pipeline

Scapy's sniff(prn=...) callback blocks the receive loop. A slow model.predict_proba() on the same thread would start dropping packets. The solution is to decouple the stages with bounded queues:

- **Thread 1 (sniff):** Scapy callback pushes IP packets into a queue. `Queue(maxsize=10000)`. If the queue is full, the packet is dropped and counted.
- **Thread 2 (infer):** pulls packets, feeds FlowBuilder, scores closed flows through Detector, and pushes AlertDecision objects into a second queue. Also calls `flush_expired()` every five seconds to emit idle-timeout flows.
- **Thread 3 (persist):** pulls AlertDecision objects and inserts them into alerts.db. Runs as a non-daemon thread so Ctrl+C waits for the queue to drain before exit.

The dashboard (Streamlit) runs in a separate process and opens alerts.db read-only using a SQLite URI (`file:alerts.db?mode=ro`). WAL mode on the writer side lets the dashboard's SELECT queries run concurrently with persist-thread INSERTs without blocking either side.

2.2 Repository layout

```
final-project/
  notebooks/           offline ML pipeline
    01_data_exploration.ipynb
    02_feature_engineering.ipynb
    03_model_training.ipynb
    04_evaluation.ipynb
  src/                 online runtime
    flow_builder.py    5-tuple flow state machine
    detector.py        RF inference + severity decision
    capture.py         3-thread pipeline entry point
    dashboard.py       Streamlit live UI
    attack_simulator.py SYN flood + port scan (loopback only)
  data/                CICIDS2017 CSVs (gitignored)
  models/              detector.pkl, metadata.json, feature_names.json
  report/              this PDF plus figures
```

3. Dataset and Feature Engineering

CICIDS2017 is a network traffic dataset published by the Canadian Institute for Cybersecurity. We used the Friday captures that contain DDoS and PortScan activity. After downloading MachineLearningCSV.zip from the CIC website and extracting the two Friday CSVs, raw row counts were 225,745 (DDoS day) and 286,467 (PortScan day).

3.1 Data quality audit

The data needed three cleaning passes:

- Column names have leading whitespace (e.g., ' Destination Port '). Stripped on load.
- Rate columns contain Infinity values where flow duration is zero. Replaced with 0.0.
- Duplicate rows exist in both files. Dropped before any train/test split.

The duplicate rate is the interesting part. The DDoS file has 2,633 duplicates (1.17%). The PortScan file has 72,353 duplicates (25.26%). One in four. This matters because port-scan flows collapse to identical feature vectors once the 5-tuple is stripped: one SYN, same attacker, similar timing, only the destination port changes. Splitting before deduplicating would put the same flow in both train and test sets, inflating the reported accuracy. Dedup-before-split kept the evaluation honest.

After cleaning, 437,207 rows remained with class balance 50% benign / 29% DDoS / 21% PortScan.

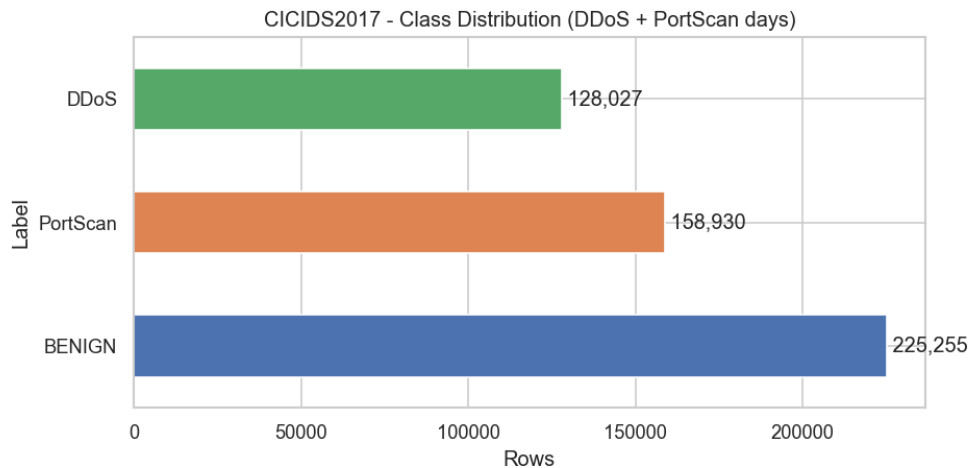


Figure 1. Class distribution across the two Friday files (post-dedup).

3.2 Feature selection

CICIDS2017 ships with roughly 78 numeric flow features from CICFlowMeter. We picked 15 across five groups:

- **Flow volume:** flow_duration, total_fwd_packets, total_backward_packets
- **Rate:** flow_bytes_per_s, flow_packets_per_s
- **Packet size:** fwd_packet_length_mean, bwd_packet_length_mean, average_packet_size
- **Inter-arrival timing:** flow_iat_mean, flow_iat_std
- **TCP flags:** syn_flag_count, ack_flag_count, fin_flag_count, rst_flag_count, psh_flag_count

We deliberately excluded destination_port. The field is tempting because port scans hit many destinations, but at inference time a benign HTTP server and a DDoS victim both use port 80. Training on destination_port risks the model learning "port 80 attacks happen in CICIDS2017" rather than a general attack-vs-benign distinction. This hygiene choice came back to bite us at inference (see Finding 3).

Feature names are locked into models/feature_names.json, the single source of truth for both notebooks and runtime. detector.py builds the feature vector by calling getattr(flow, name) for each name in that file, failing loudly if the FlowRecord is missing a field. This contract eliminated a whole class of silent-drift bugs during development.

4. Model Training and Evaluation

The classifier is a scikit-learn RandomForestClassifier with 100 trees, class_weight='balanced', and no hyperparameter tuning. Random Forest is tree-based and scale-invariant, so the runtime ships no StandardScaler. Training uses a stratified 70/15/15 train/val/test split with random_state=42 for reproducibility.

Training takes 2.92 seconds on the laptop. Validation macro F1 is 0.9997. The held-out test set reaches 99.96% accuracy and 0.9996 macro F1. Per-class F1: benign 0.9996, DDoS 0.9995, PortScan 0.9998.

These numbers are a known ceiling for CICIDS2017. Published benchmarks report similar accuracy because the attacks were captured in a controlled lab environment with clean signatures. The take-away is not "the model is excellent" but "the model learned the general shape of these attacks."

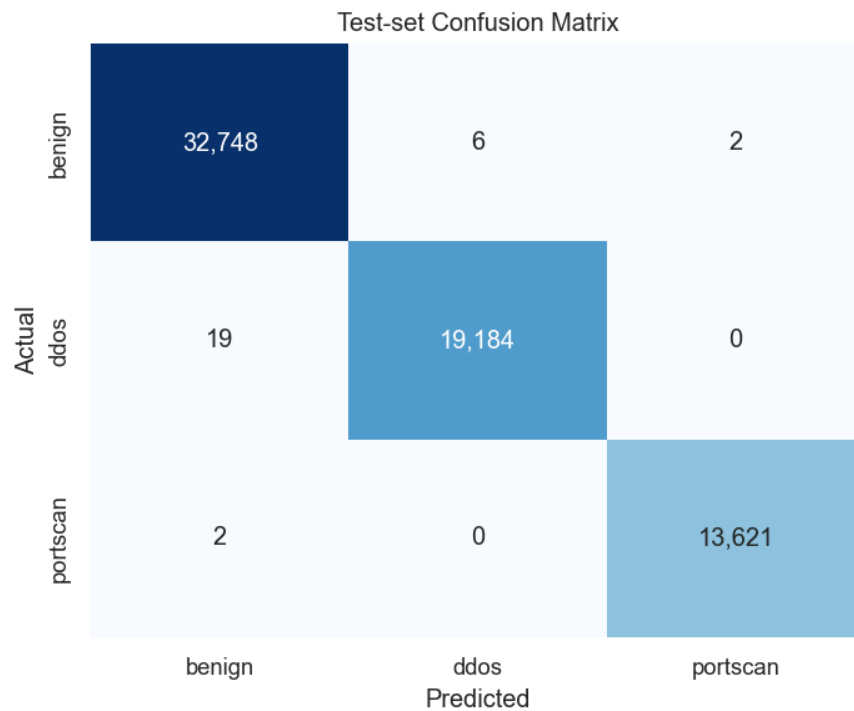


Figure 2. Test-set confusion matrix. Off-diagonal counts are in the 0-19 range out of ~65,000 samples.

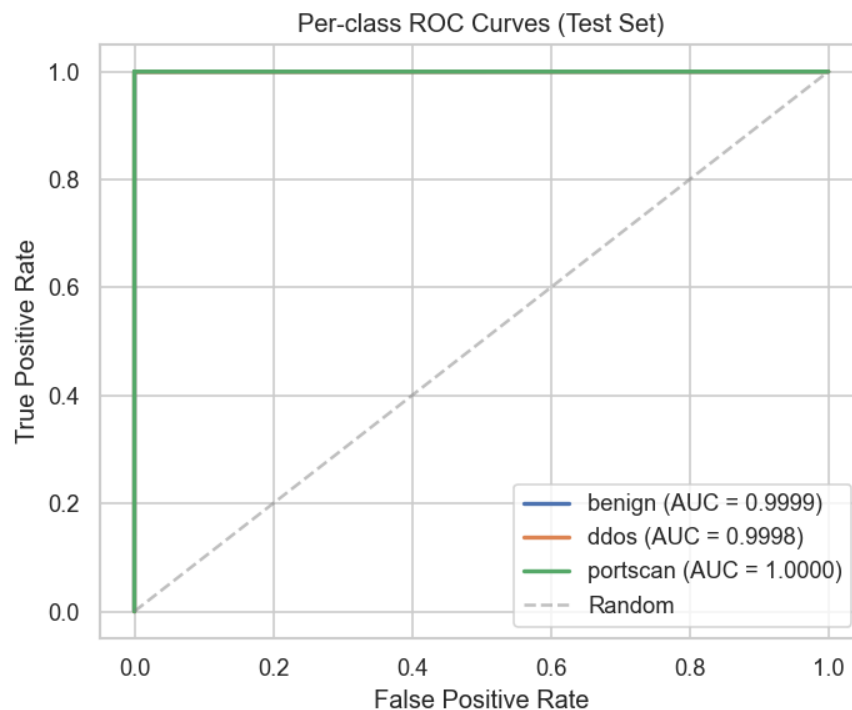


Figure 3. One-vs-rest ROC curves per class on the test set. All three AUCs round to 1.0000.

4.1 Threshold calibration

At runtime, detector.py flags a flow when $\max(P(\text{ddos}), P(\text{portscan}))$ exceeds a threshold. The threshold is calibrated against the validation set's benign flows at a target false-positive rate. At FPR 0.001 (one benign flow in a thousand crossing the line), the calibrated threshold was 0.1322.

In the offline evaluation this threshold preserved 99.98% recall on both DDoS and PortScan attack classes. Recall was calculated as the fraction of real attack flows whose attack_probability sat above the threshold.

5. Network Protocols and Runtime

The runtime exercises several network-programming concepts from the course syllabus. This section ties the code to the concepts.

5.1 Packet capture and dissection

Scapy's sniff() operates below the TCP/IP stack, receiving raw Ethernet frames from Npcap on Windows. flow_builder.py parses each frame and extracts the 5-tuple plus TCP flag bitmask:

```
def _parse(self, pkt):
    if IP not in pkt:
        return None
    ip_layer = pkt[IP]
    if TCP in pkt:
        t = pkt[TCP]
        return (ip_layer.src, int(t.sport),
                ip_layer.dst, int(t.dport),
                "tcp", len(pkt), int(t.flags))
    if UDP in pkt:
        u = pkt[UDP]
        return (ip_layer.src, int(u.sport),
                ip_layer.dst, int(u.dport),
                "udp", len(pkt), 0)
    return None
```

The fields we read directly are Ethernet encapsulation (the frame delivered by Npcap), the IP layer (source/destination, protocol number, length via len(pkt)), and the TCP or UDP headers (source/destination port and, for TCP, the flag bitmask with SYN=0x02, ACK=0x10, FIN=0x01, RST=0x04, PSH=0x08).

5.2 5-tuple flow aggregation

A flow is the NetFlow-style aggregation of packets sharing a 5-tuple. flow_builder.py keys a dict with FlowKey dataclasses that normalise direction so packets from A to B and B to A map to the same key. The first packet seen defines "forward"; subsequent packets are tagged forward or backward by comparing the packet's source against the stored initiator. Running statistics (mean and standard deviation) use Welford's algorithm so memory stays $O(1)$ per flow regardless of packet count.

5.3 TCP lifecycle encoded in the close logic

A flow closes for one of three reasons: RST (abrupt), FIN from both sides (graceful), or idle timeout (no packets for 60 seconds). The priority is RST > FIN > timeout, because the more abrupt reason carries

more information about what happened on the wire.

```
def should_close_flow(state, direction, saw_fin, saw_rst,
                      now_mono, idle_timeout_s=60.0):
    if saw_rst:
        return 'rst'
    if saw_fin:
        if direction == 'fwd':
            state.fwd_fin_seen = True
        else:
            state.bwd_fin_seen = True
    if state.fwd_fin_seen and state.bwd_fin_seen:
        return 'fin'
    if (now_mono - state.last_pkt_ts_mono) > idle_timeout_s:
        return 'timeout'
    return None
```

The FIN logic encodes TCP's four-way handshake. A single FIN is not enough to close: the other side may still have data to send. The function updates `fwd_fin_seen` or `bwd_fin_seen` when a FIN arrives, then checks whether both are set on each subsequent packet.

5.4 SQLite with WAL mode

The `persist` thread opens `alerts.db` with `PRAGMA journal_mode=WAL` and `PRAGMA synchronous=NORMAL`. WAL mode (Write-Ahead Log) separates writes into a sidecar file, letting readers see a consistent snapshot without blocking writers. This is what lets the dashboard re-query the database once per second while `capture.py` is inserting alerts, all without explicit coordination.

6. Attack Simulation and Demo

`attack_simulator.py` generates two kinds of traffic for the demo. Both use Scapy to craft raw TCP packets.

- **syn_flood:** N SYN packets to a single destination port with random source ports. Each packet becomes a distinct 5-tuple, so the capture pipeline sees many short flows. The default count is 500 at 50 pps.
- **port_scan:** one SYN to each port in a range with a fixed source port. Each destination port is its own flow. Default range is 20-1024, rate 50 pps.

The simulator refuses any target that is not in `127.0.0.0/8` or `::1`. The check is a hard exit with a non-zero return code, not a warning, to prevent accidental off-host sends.

Rate limiting uses a drift-corrected pacer. Given a target rate, the pacer schedules `next_time = now + 1/rate` and sleeps until that moment. Windows sleep resolution is roughly 10 ms, so individual sleeps can overshoot; the drift-corrected scheme keeps the average rate close to the target over a batch.

On Scapy 2.5, Windows loopback sends go through `L3pcapSocket` (Npcap) automatically, provided Npcap was installed with "Support loopback traffic" enabled. Older tutorials recommend `conf.L3socket = L3RawSocket`; that class was removed from Scapy 2.5's Windows arch module and the override is neither needed nor compatible.

7. Findings

Three findings came out of end-to-end testing that do not appear in the textbook version of this project.

7.1 Packet-size features dominate; TCP flag counts are nearly useless

When we picked features, we chose five TCP flag counts expecting them to carry the attack signal. Random Forest feature importance says otherwise.

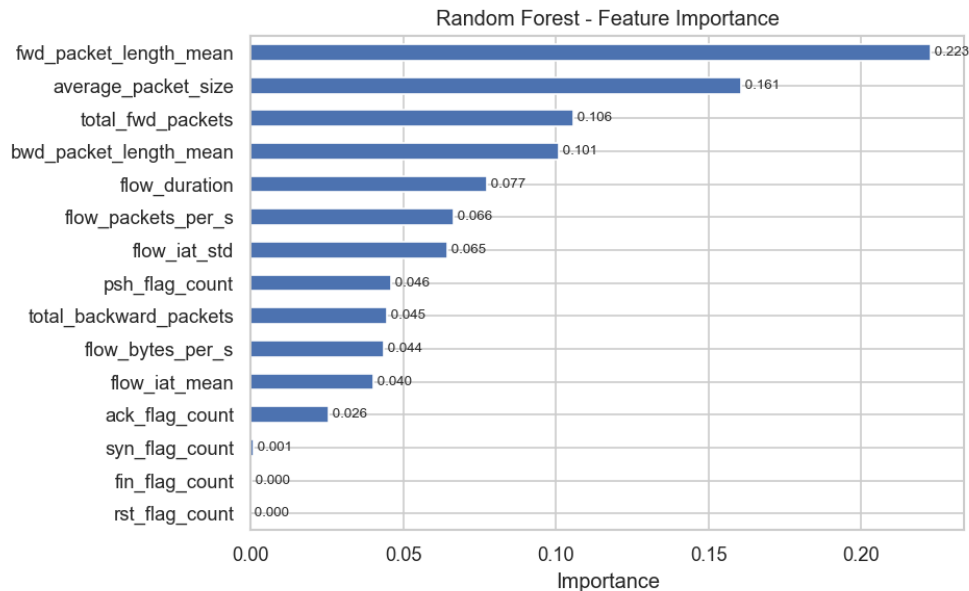


Figure 4. Random Forest feature importances from the trained detector. Packet-size features dominate.

Top five by importance: fwd_packet_length_mean (0.2228), average_packet_size (0.1608), total_fwd_packets (0.1057), bwd_packet_length_mean (0.1008), flow_duration (0.0773). Bottom three: rst_flag_count (0.0000, literally zero), fin_flag_count (0.0002), syn_flag_count (0.0011).

Why? CICFlowMeter aggregates per flow over tens of packets. Raw SYN counts get diluted: a 100-packet flow that starts with a SYN has the same "SYN per packet" ratio as any TCP flow that starts with a SYN. What the model learned is simpler: attack packets are small and uniform. DDoS SYN packets are 60 bytes. Port scan packets are 60 bytes. Benign HTTP traffic varies widely in packet size. Size tells the story; flag counts turn out to be redundant once size is in the model.

We kept the low-importance features rather than pruning them. Retraining for a marginal simplification in flow_builder.py would have churned the model and metadata without measurable benefit. The importance values themselves are useful content for this report.

7.2 Domain shift required runtime threshold recalibration

The calibrated threshold (0.1322) was computed against CICIDS2017 validation benign. When we ran the full pipeline on Windows loopback with our synthetic attacks, the attack flows produced an attack probability around 0.08. That's well above the ~0% we saw for CICIDS2017 benign, but still below 0.1322, so nothing raised an alert.

Two environmental differences explain the gap. First, loopback port scans get RST responses from the Windows kernel, giving bwd_packet_length_mean around 40 bytes. In CICIDS2017 the scanned

targets sometimes dropped packets silently, producing `bwd_packet_length_mean` closer to zero. The model learned "non-zero backward length is not a scan" and applies that rule against our traffic. Second, loopback inter-arrival times are measured in microseconds; CICIDS2017 was captured on real hardware with real network delays. The timing features fall into ranges the model never saw during training.

For the demo we lowered the runtime threshold to 0.05 (keeping the calibrated 0.1322 in `metadata.json` under `alert_threshold_calibrated` for audit) and switched the severity function to flag purely on attack probability rather than gating on the model's argmax class:

```
def _decide_severity(self, predicted_class, attack_probability):
    if attack_probability >= self._threshold:
        return "alert"
    return "info"
```

Every real NDR deployment faces this choice: calibrate on lab data, retune on production traffic. This project ran into the problem at a small scale, and the fix is the standard one.

7.3 Flow-level features cannot distinguish intent

With the runtime threshold at 0.05, the pipeline reports around 20 alerts per 30 seconds on idle loopback, before any attack simulator runs. The culprits are Windows services repeatedly probing 127.0.0.1 TCP ports that are not listening. Each probe sends one SYN, gets an RST back from the kernel, and closes the flow. That is exactly the packet pattern our port scanner produces.

The model is not wrong. It is reporting that certain flows look like scans. Whether one is a legitimate IPC probe and the other is malicious is information that does not live in the feature vector. Real NDR deployments rely on additional signals the system does not have: the process that opened the socket, reputation of the source, frequency and pattern of prior attempts. This project is a flow classifier, not an intent classifier.

A partial mitigation for the demo is a BPF filter at capture time, excluding the most chatty local port so that the "idle" state of the dashboard genuinely reads idle. The finding goes in the report regardless: it is more honest to document the limitation than to hide it behind a filter.

8. Challenges

The short list of things that required time to resolve, roughly in order:

- **CICIDS2017 data quality.** Whitespace-prefixed column names, Infinity in rate columns, and 25% duplicates in the PortScan file. Handled in `02_feature_engineering.ipynb`.
- **Feature-name contract.** The names and order of features between training and runtime must match exactly, or the model silently predicts garbage. Solved by making `feature_names.json` the single source of truth and validating on every predict call.
- **Windows setup.** Npcap has to be installed with "Support loopback traffic" enabled or sniffing 127.0.0.1 does not work. Each Administrator PowerShell has to re-activate the venv, because elevation does not inherit environment variables.
- **Scapy 2.5 API changes.** Older tutorials recommend setting `conf.L3socket = L3RawSocket` for Windows loopback. Scapy 2.5 removed that class. The default `L3pcapSocket` handles loopback via Npcap without any override.

- **Sniff interface selection.** Without `--iface`, Scapy auto-selects the default network adapter (Wi-Fi in our case), not the loopback adapter. Packets from `attack_simulator.py` were never captured. Fix was to pass `--iface "\Device\NPF_{Loopback}"`.
- **Distribution shift.** Covered in Finding 7.2.
- **Schema migration.** Adding a column to `alerts.db` mid-project required either deleting the old database or adding an ALTER TABLE fallback in `capture.py`. We chose the second option and wrote a short migration registry that runs on startup.

9. Conclusion

The offline pipeline reaches textbook accuracy on CICIDS2017 because the dataset is well-behaved and the attack signatures are clean. The runtime pipeline works end-to-end: Scapy captures packets on the loopback adapter, a 5-tuple flow builder emits FlowRecords on FIN, RST, or timeout, the detector scores them through the trained model, and the Streamlit dashboard shows alerts in real time with red-highlighted rows when `attack_simulator.py` fires.

The project's more useful output is the three findings in section 7. Feature importance does not always match intuition. Calibration drifts when the environment changes. Flow-level statistics cannot separate malicious probing from legitimate IPC probing. None of these are catastrophic. All three are the kind of thing an engineer deploying an NDR system in production will run into, and writing them down is the point.

References

CICIDS2017 dataset. Canadian Institute for Cybersecurity, University of New Brunswick. <https://www.unb.ca/cic/datasets/ids-2017.html>

Scikit-learn developers. RandomForestClassifier documentation. <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html>

Scapy project documentation. <https://scapy.net/>

Streamlit documentation. <https://docs.streamlit.io/>

SQLite Write-Ahead Logging. <https://www.sqlite.org/wal.html>

Npcap packet capture library. <https://nmap.org/npcap/>